

EXPLAINABLE AUTONOMOUS GAME AGENTS USING DEEP REINFORCEMENT LEARNING AND RETRIEVAL-AUGMENTED GENERATION (RAG)

Sourish Dutta and Prathamesh Chalak

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

Corresponding author: Sourish Dutta (e-mail: sourish.dutta2024@vitstudent.ac.in). Team member: Prathamesh Chalak.

This work is being carried out as part of the undergraduate project at Vellore Institute of Technology, Vellore, India.

ABSTRACT Modern reinforcement-learning agents can now play games at a fairly competent level, but the policies driving them are essentially black boxes — a trained network can pick an action without being able to say why. This project builds a two-dimensional combat arena in which an agent is trained with Proximal Policy Optimization (PPO), and pairs it with an explanation layer built on Retrieval-Augmented Generation (RAG), so that a human evaluator can pause the simulation and ask, in plain English, why the agent did what it did. Instead of feeding raw telemetry straight into a language model, which either overflows the context window or leads to hallucinated answers, the system logs environment state together with the actor-critic's value estimates and action-probability distributions into a vector database, and retrieves only the slice of that log relevant to a given question to ground each explanation. This report covers the first-phase design of the system: the motivation behind it, the literature it builds on, the system architecture, and the dataset and method used to implement it.

INDEX TERMS Deep reinforcement learning, explainable AI, retrieval-augmented generation, Proximal Policy Optimization, actor-critic networks, vector databases, game simulation.

I. INTRODUCTION

A. OVERVIEW

Game-playing agents trained with Deep Reinforcement Learning (DRL) can now learn fairly sophisticated, human-competitive tactics just from a reward signal, with no hand-coded rules at all. The catch is that this comes at the cost of transparency. The policy network deciding what the agent does next is a dense, high-dimensional function approximator, and its internal computations stay opaque even to the people who trained it. This project, “Explainable Autonomous Game Agents Using Deep Reinforcement Learning and Retrieval-Augmented Generation (RAG),” tries to close that transparency gap. We build a real-time, two-dimensional competitive combat arena where an autonomous agent is trained end-to-end using Proximal Policy Optimization (PPO), and we pair this agent with an explainability layer built on Retrieval-Augmented Generation. This layer lets a human evaluator pause the simulation at any point and ask, in plain English, why the agent behaved the way it did, and get back an answer that is

grounded in the agent's actual recorded internal state rather than a plausible-sounding guess.

The system brings together two branches of AI that are usually kept separate: sub-symbolic, gradient-trained neural policies (the DRL agent) and symbolic, language-based reasoning (the RAG-driven explanation engine). We connect the two through a telemetry pipeline and a vector database, which keeps them loosely coupled — the agent trains and acts with no awareness of the explanation layer at all, while the explanation layer only ever reasons over facts that were actually logged during play.

B. PROBLEM STATEMENT

Deep Reinforcement Learning algorithms such as PPO and Deep Q-Networks (DQN) are very good at complex sequential decision-making in interactive environments, but they operate as black boxes — given a high-dimensional observation vector, they output an action with no explanation attached. In both safety-critical settings and academic evaluation, understanding why an agent chose a

particular action can matter just as much as the outcome of that action. The obvious fix — just feeding raw frame-by-frame telemetry straight into a general-purpose LLM — doesn't really work in practice, for two reasons. First, even a short gameplay session generates far more numerical state data than fits comfortably inside an LLM's context window. Second, when an LLM is asked to reason about data it doesn't actually have in front of it, it tends to hallucinate explanations that sound plausible but aren't true. This project deals with the problem by narrowing down what the LLM is asked to reason over: telemetry is written into a vector database as it's generated, and a RAG pipeline retrieves only the small, precisely relevant slice of that telemetry needed to answer a specific question, so the LLM's output stays grounded in verifiable fact.

C. OBJECTIVES

- Design and build a custom 2D competitive arena using Pygame, wrapped in the Farama Gymnasium API, suitable for training a reinforcement-learning agent.
- Train a tactical decision-making agent with PPO via Stable-Baselines3, able to progress from random exploration to competent combat tactics such as covering, retreating and engaging.
- Build a frame-level telemetry logging pipeline that captures not just environment state but also the agent's internal policy signals — value estimates, action-probability distributions and actor-critic latent features — at each important decision point.
- Build a Retrieval-Augmented Generation pipeline over a vector database (ChromaDB) that can pull out the exact telemetry relevant to a natural-language question and pass it to a local or hosted LLM for grounded answer generation.
- Extend the explanation pipeline past simple state description into counterfactual reasoning — explaining not just what the agent did, but why the alternatives it didn't pick were less attractive at that moment, using the logged value and probability signals.
- Evaluate the system on three fronts: RL training convergence (via TensorBoard), retrieval accuracy and latency of the vector database, and factual faithfulness of the generated natural-language explanations.

D. SIGNIFICANCE OF THE WORK

This work matters beyond just building an interpretable game bot. Explainability is basically a prerequisite for trusting any autonomous decision-making system, and games give us a controlled, low-risk sandbox to develop and

test explainability techniques before pushing them into higher-stakes domains. A tactical agent that can justify its own decisions in plain language is directly useful in game design and balancing (developers can ask why an AI keeps exploiting a certain mechanic), in AI education (students can question a trained policy to build intuition about reinforcement learning), and more broadly in domains that share the same decision structure as a combat arena — military training simulations and autonomous robotic systems, for instance — where an operator being able to ask “why did you do that?” and get a truthful, grounded answer is central to safe deployment. By showing that a RAG-based approach can give hallucination-resistant, sub-second explanations for a live DRL agent, this project offers a template that should generalise to any interactive autonomous system whose internal state can be logged.

E. MODIFICATIONS INCORPORATED BASED ON ABSTRACT-STAGE FEEDBACK

Reviewer Feedback Received

“The use of already implemented benchmark techniques should be avoided wherever possible... Q-Value / Advantage-Aware Explanation RAG: Log not only state environment variables but also the PPO Actor-Critic latent feature representations, value estimations $V(s)$, and action probability distributions $\pi(a|s)$. This allows the RAG system to ground its explanations in why alternative actions were rejected (counterfactual reasoning).”

The abstract we submitted at the proposal stage described a telemetry pipeline that only logged externally observable environment variables — health, position, ammunition, and the action selected. That's enough to describe what happened, but a system built purely on that kind of logging can only restate observable facts; it can't explain why one action was preferred over another. This is a fairly standard, already-established use of RAG for log-grounded question answering, and on its own it doesn't really count as a novel contribution. In direct response to the reviewer's feedback, we revised the design so that the telemetry logger now also captures, at every logged decision point, three internal signals produced by the PPO actor-critic network:

- The state-value estimate $V(s)$ — the critic network's prediction of expected future return from the current state.
- The full action-probability distribution $\pi(a|s)$ over the discrete action set, not just the action that ended up being sampled or selected.

- The actor-critic's penultimate-layer latent feature vector, a compressed representation of how the network internally “saw” the state before producing its policy and value outputs.

With these three signals stored alongside the environmental state in ChromaDB, the RAG engine can now be asked not just “what did the agent do at 00:42?” but “why did the agent retreat instead of shooting at 00:42?” The retrieval step surfaces the probability the policy assigned to each rejected action along with the value the critic associated with the state, and the LLM is prompted to explicitly compare these figures, producing a counterfactual explanation (for example: “shooting was assigned only 8% probability because the critic valued the current position at -0.34 given the agent's low health, making retreat the dominant choice”) that's grounded in numbers the network itself produced, not in the LLM's own guesswork. We're treating this modification as a core architectural change rather than an optional extra — it shows up in the design figure (Section III), in the Module 2 and Module 3 descriptions (Section IV), and in the Method Description (Section V-B), and none of the originally proposed benchmark-only logging scheme has been kept without this extension.

II. LITERATURE REVIEW

This section looks at the four bodies of prior work this project draws on and builds against: deep reinforcement learning for game-playing agents, explainable AI (XAI) techniques for black-box policies, retrieval-augmented generation for grounding language models in external facts, and existing attempts specifically at explaining reinforcement-learning behaviour. Each subsection ends by pointing out the specific gap that this project's design tries to address.

A. DEEP REINFORCEMENT LEARNING IN GAME ENVIRONMENTS

Deep reinforcement learning has been the dominant approach for training agents in interactive game environments ever since value-based methods like Deep Q-Networks (DQN) showed human-level performance on Atari games using nothing but raw pixel observations and a scalar reward signal. Policy-gradient methods later improved on the stability and sample efficiency of value-based approaches; PPO in particular has become something of a default choice for continuous-control and tactical decision-making tasks, because it limits the size of each policy update and avoids the kind of destructively large updates that can destabilise earlier policy-gradient methods. Actor-critic

architectures, which PPO uses internally, keep two coupled estimators — a policy (actor) mapping states to action probabilities, and a value function (critic) estimating expected return — and it's exactly this critic output and the actor's probability distribution that this project's telemetry pipeline exposes to the explanation layer, rather than treating them as internal-only quantities that get thrown away after each training step.

Within game AI specifically, prior work has used DRL to train agents for combat, navigation and resource-management tasks in custom or simplified arena environments, but usually only reports aggregate performance metrics like win rate or cumulative reward. What's largely missing from this literature is any real mechanism for inspecting a trained agent's decision at a single instant in a way a non-technical evaluator could actually use — performance curves tell you how well an agent plays on average, not why it made one specific choice at one specific frame.

B. EXPLAINABLE AI (XAI) FOR BLACK-BOX REINFORCEMENT LEARNING POLICIES

XAI research on reinforcement learning has mostly gone in two directions. Saliency-based methods borrow techniques from supervised deep learning to highlight which parts of a visual observation most influenced an action, producing heatmaps over game frames. Policy-distillation and interpretable-surrogate methods instead train a simpler, human-readable model (a decision tree, say) to approximate the trained neural policy's behaviour, trading off fidelity for interpretability. Both families of technique are really aimed at machine-learning practitioners — a heatmap or a decision tree — rather than producing a natural-language explanation an ordinary end user could actually use. Neither approach on its own answers a question phrased in plain English, and neither naturally supports the kind of counterfactual query (“why not the other action?”) this project is targeting.

C. RETRIEVAL-AUGMENTED GENERATION AND GROUNDED LANGUAGE MODELS

Retrieval-Augmented Generation was introduced as a way to combine the fluent generation ability of large language models with the factual reliability of an external, queryable knowledge source: instead of relying only on facts memorised during pretraining, a RAG system embeds an incoming query, retrieves the most semantically similar documents from a vector database, and conditions the language model's generation on that retrieved context. This is now a well-established architecture for open-domain question answering and document-grounded chat, where

it's been shown to noticeably cut down on hallucination compared to an ungrounded LLM answering purely from memory. Applying it here means pointing it at a very different kind of data — structured, numerical, time-stamped simulation telemetry — rather than the prose documents RAG is usually applied to, which forces some deliberate choices about how numerical state gets serialised into text before embedding, and about what granularity of chunk (a single frame vs. a decision window) gets retrieved per query.

D. EXISTING APPROACHES TO EXPLAINING REINFORCEMENT-LEARNING AGENT BEHAVIOUR IN NATURAL LANGUAGE

A smaller body of work has looked at generating natural-language explanations of RL agent behaviour directly, usually by filling in pre-written explanation templates using the current state (for example, filling a fixed sentence template with the agent's health and the enemy's distance). Template-based approaches are fast and grounded by construction, but they can't answer open-ended or comparative questions that weren't anticipated when the template was written, and they scale badly as the number of possible situations grows. Using an LLM conditioned on retrieved telemetry, like this project does, keeps the grounding guarantees of a template system (the model only ever sees facts pulled from an actual log entry) while still allowing the free-form, conversational question answering a fixed template can't support.

E. RESEARCH GAP ADDRESSED BY THIS PROJECT

Putting the four strands above together: the literature gives us agents that play well but can't explain themselves (Section II-A), explanation techniques aimed at engineers rather than end users (Section II-B), a grounding architecture that's proven on prose but not really adapted to simulation telemetry yet (Section II-C), and natural-language explanation systems that are either template-limited or ungrounded (Section II-D). This project's contribution is to combine RAG-based grounding with reinforcement-learning-specific internal signals — the critic's value estimate and the actor's full action-probability distribution — so the resulting explanations end up being both open-ended (any question can be asked, not just the ones a template anticipated) and genuinely counterfactual (they can say why rejected actions were rejected, not just what was chosen). This is the specific gap the reviewer feedback in Section I-E pointed at.

III. DESIGN FIGURE

Figure 1 shows the overall system architecture. The design follows a three-module pipeline where data flows strictly left-to-right / top-to-bottom: the Game Environment module produces an observation, the Reinforcement Learning module consumes that observation and returns an action along with its internal actor-critic signals, and the Telemetry & RAG Engine records both the environment state and the actor-critic signals so they can later be retrieved to answer a natural-language question.

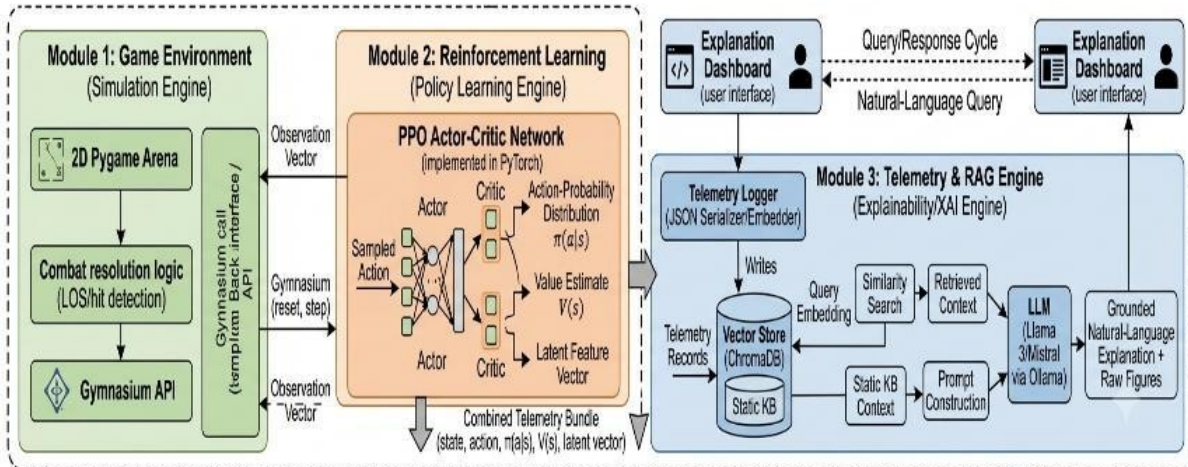


FIGURE 1. Overall Architecture of the Explainable Game Agent System — Game Environment → Reinforcement Learning → Telemetry & RAG Engine, with a link to the Explanation Dashboard.

Explanation of the Design Figure

At the top of the figure, the Game Environment module renders the 2D arena and works out each entity's position, health, and combat outcome on every simulation tick,

exposing all of this as a structured observation vector through the Gymnasium `reset()/step()/render()` interface. This observation flows into the Reinforcement Learning module, where the trained PPO actor-critic network consumes it and

produces three outputs at once: the sampled action sent back to the environment to be executed, the action-probability distribution $\pi(a|s)$ over all candidate actions, and the critic's scalar value estimate $V(s)$ for the current state. All three outputs — not just the sampled action — get passed down to the third module.

The Telemetry & RAG Engine receives this combined package on every logged tick or trigger event and does two jobs. First, its logger serialises the environment state, the chosen action, the full probability distribution, and the value estimate, together with the actor-critic's latent feature vector, into a structured JSON record, and writes an embedding of that record into ChromaDB alongside a static knowledge base of game rules and reward weights. Second, when a user submits a natural-language query through the Explanation Dashboard, the same module embeds the query, runs a similarity search against the vector store to retrieve the most relevant frame(s), and passes the retrieved numerical context to the LLM, which sends a grounded natural-language explanation back to the dashboard. The two Explanation Dashboard boxes in Figure 1 represent the same interface at the two ends of this query/response cycle — a query going out and an explanation coming back — rather than two separate dashboards; a future redraw of this figure could collapse them into a single box with one bidirectional arrow to Module 3 to avoid any ambiguity on a first read. This cycle can happen at any point after a frame has been logged, including while the simulation is paused.

IV. MODULE DESCRIPTION

The system is made up of three tightly integrated but independently testable modules. Each one is described below in terms of its purpose, its internal functionality, and how it interfaces with the other two.

A. MODULE 1 — GAME ENVIRONMENT (SIMULATION ENGINE)

Purpose To provide a controllable, deterministic-when-seeded, real-time 2D arena in which a tactical combat scenario can be simulated and rendered, and which exposes a standard reinforcement-learning interface so any Gymnasium-compatible training algorithm can be trained against it without needing environment-specific glue code.

Functionality Renders a bounded 2D arena with Pygame, including static obstacles, at least two agent-controlled entities, and a HUD showing health and ammunition. Implements combat resolution logic — line-of-sight / hit detection between entities, damage application, and win/loss/draw termination conditions. Wraps the simulation in the Farama Gymnasium API, exposing `reset()` to start an episode, `step(action)` to advance one tick and return (observation, reward, terminated, truncated, info), and `render()` to draw the current frame. Builds the observation vector fed to the RL module: relative coordinates to the opponent, ray-cast distances to the nearest walls/obstacles, current health ratio, ammunition count, and a small history buffer of recent relative movement. Figure 2 shows a conceptual mock-up of how this arena and HUD are intended to look once built — not an actual in-engine screenshot, since Module 1 has not been implemented yet at this first-phase stage.

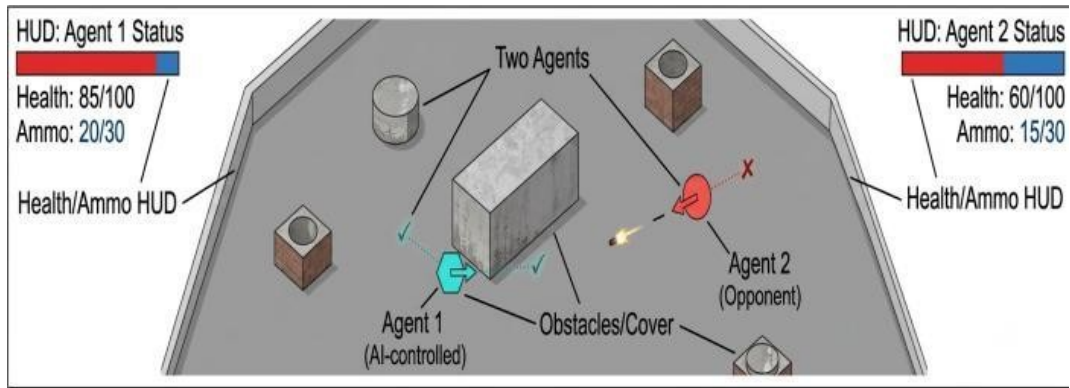


FIGURE 2. Conceptual mock-up of the planned 2D arena layout and HUD, illustrating the two agents, obstacles, and the intended health/ammo display. This is a design mock-up, not a captured screenshot — an actual rendered image will replace it once Module 1 has been built and run.

Interaction with Other Modules Module 1 exposes its Gymnasium-standard `step()/reset()` interface to Module 2, which drives the environment forward one action at a time during both training and inference. It also exposes a lightweight state-snapshot callback that Module 3 consumes immediately after every `step()` call, so the telemetry logger

always records the state that resulted from the most recently executed action.

B. MODULE 2 — REINFORCEMENT LEARNING (POLICY LEARNING ENGINE)

Purpose To learn, through trial-and-error interaction with Module 1, a policy that selects tactically sound actions

(move, shoot, take cover, retreat) from the observation vector, and — per the design change described in Section I-E — to expose its internal actor-critic reasoning at inference time rather than just its final action choice.

Functionality Implements the actor-critic network in PyTorch, trained with PPO via Stable-Baselines3, chosen for its update stability compared to plain policy-gradient methods and its fit for the mixed discrete-action, dense-reward setting of the arena. The actor head outputs a probability distribution $\pi(a|s)$ over the discrete action set {Move North/South/East/West, Shoot, Take Cover, Retreat}; the critic head outputs a scalar value estimate $V(s)$. During both training and evaluation rollouts, the module records — for every single step, not just logged telemetry events — the sampled action, the full $\pi(a|s)$ vector, the scalar $V(s)$, and the 128-dimensional latent activation vector from the actor-critic's shared penultimate layer, then forwards this bundle to Module 3. It also supports checkpointing at fixed training-step intervals (10k, 50k, 100k, 500k steps, for example) so agents at different skill levels can be compared side by side during evaluation, per the project's human-in-the-loop evaluation objective.

Reward Function Design The reward function combines a sparse terminal component (a large positive reward for winning an episode, a large negative one for losing) with dense shaping terms that push the agent toward tactically sound intermediate behaviour: a positive reward for landing a hit, a negative reward for taking damage, a smaller positive reward for holding a defensible position when at low health, and a small time penalty per step to discourage stalling. This shaped reward is what the critic's $V(s)$ ultimately learns to predict, which is why $V(s)$ ends up being meaningful enough to put directly into explanations — a low $V(s)$ at a given frame is a direct, human-checkable sign that the network itself thought the situation was unfavourable. The specific weight values shown in Figure 3 (+10.0 / -10.0 for the terminal outcome, +0.5 / -0.2 / +0.3 / -0.01 for the shaping terms) are our starting point for implementation, not tuned final values — they still need to be adjusted empirically once training actually begins, most likely by watching whether the agent over-prioritises defensive play or reckless engagement and rebalancing accordingly.



FIGURE 3. Breakdown of each reward-shaping term and its (illustrative, pre-tuning) weight.

Interaction with Other Modules Module 2 consumes observations from Module 1 through the Gymnasium step() interface and returns the sampled action for execution. At the same time, it sends the actor-critic bundle (action, $\pi(a|s)$, $V(s)$, latent vector) to Module 3 through an in-process callback, avoiding any file or network round-trip during training so logging doesn't measurably slow down the training loop.

C. MODULE 3 — TELEMETRY & RAG ENGINE (EXPLAINABILITY / XAI ENGINE)

Purpose To keep a queryable, factual record of everything the agent observed and internally computed, and to answer natural-language questions about that record — including counterfactual questions about rejected actions — without letting the underlying LLM hallucinate facts that aren't actually in the log.

Telemetry / Data Serialization Sub-module Captures structured JSON telemetry at a fixed sampling interval, plus at trigger events (taking damage, choosing to retreat, eliminating an opponent, health dropping below a low-health threshold). Logged fields, per the revised design: timestamp, agent health, opponent position, the action selected, the full action-probability distribution $\pi(a|s)$, the critic's value estimate $V(s)$, the actor-critic latent feature vector, the immediate reward received, and active environmental flags such as is_low_health. Each record is serialised, a text/vector embedding is computed for similarity search, and both the record and its embedding are written into ChromaDB (with Qdrant kept as an alternative backend for larger-scale deployment).

RAG / Explainability Sub-module On receiving a user question (for example, "Why did the AI retreat at 00:42 instead of attacking?"), this sub-module embeds the query and runs a vector similarity search against the telemetry store

to retrieve the exact frame(s) around the referenced timestamp. It also pulls relevant entries from the static knowledge base of game mechanics, weapon stats, and reward weights, so the LLM has access to the rules of the game and not just the moment-specific numbers. It builds a prompt that explicitly lays out the retrieved $V(s)$, $\pi(a|s)$ values for the chosen and rejected actions, and the relevant reward weights for the LLM (Llama 3 / Mistral via Ollama, or the OpenAI API), instructing it to compare the value and

probability of the chosen action against the alternatives before answering — this is the mechanism that actually produces the counterfactual reasoning. It then returns a natural-language answer to the Explanation Dashboard, along with the raw retrieved figures, so an evaluator can independently check the explanation against the source numbers. Figure 4 walks through this whole sequence step by step.

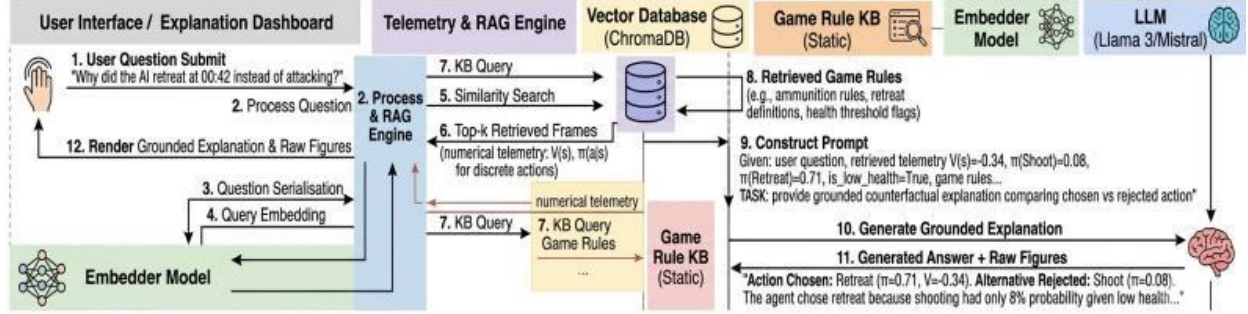


FIGURE 4. Sequence flow: user question $\rightarrow$ query embedding $\rightarrow$ ChromaDB similarity search $\rightarrow$ retrieved frame(s) + rule KB $\rightarrow$ prompt construction $\rightarrow$ LLM $\rightarrow$ natural-language answer + raw figures shown on dashboard.

Interaction with Other Modules Module 3 subscribes to the per-step callback exposed jointly by Modules 1 and 2, giving it a consistent, synchronised view of environment state and actor-critic internals for every frame it chooses to log. It exposes a query interface consumed by the Explanation Dashboard (built with Streamlit or a custom GUI), which is the only user-facing part of the whole system.

V. DATASET AND METHOD DESCRIPTION

A. DATASET DESCRIPTION

This project doesn't rely on a pre-existing external dataset — the primary dataset is a self-generated telemetry corpus produced by the simulation itself as the agent trains and plays. This is a deliberate choice: since the explanation engine has to be able to trace every claim it makes back to a specific, verifiable log entry, the dataset and the environment producing it need to be fully under the project's control.

- Source: generated at runtime by Module 1 (environment state) and Module 2 (actor-critic signals), captured and serialised by Module 3.
- Size: telemetry keeps accumulating during training and evaluation; a typical evaluation session of a few thousand simulation steps should give somewhere on the order of several thousand to tens of thousands of logged frame records, depending on the sampling interval and how often trigger events happen.
- Features per record: timestamp, agent health, opponent relative position, selected action, full action-probability distribution $\pi(a|s)$, critic value estimate $V(s)$, actor-critic

latent feature vector, immediate reward, and boolean environmental flags (e.g. `is_low_health`, `is_in_cover`).

- A secondary, static dataset made up of the game's own mechanics — weapon damage values, movement speeds, and the explicit reward-shaping weights from Section IV-B — is written once by the team and stored in the same vector database, so the LLM can ground explanations in the rules of the game and not only in moment-specific numbers.
- Preprocessing: numerical telemetry records are turned into a compact textual representation (a short structured sentence per record) before embedding, since the embedding model is text-based; latent feature vectors are stored as raw floating-point arrays alongside the text representation, so exact values stay available to the LLM prompt even though they aren't embedded directly.

B. METHOD DESCRIPTION

The overall method is organised into four stages, matching the four sub-systems described in Section IV, plus an evaluation stage. Each stage's approach, and the reasoning behind the specific technique picked, is described below.

Reinforcement Learning Methodology The agent is trained with Proximal Policy Optimization, an on-policy actor-critic algorithm, implemented via Stable-Baselines3 on top of PyTorch. We picked PPO over a purely value-based method like DQN because the arena's action space mixes discrete tactical choices in a way that benefits from PPO's stable, clipped policy updates, and because PPO naturally maintains the two internal signals — the actor's probability distribution

and the critic's value estimate — that the revised design (Section I-E) needs for counterfactual explanation; a value-based method would need extra engineering to expose an equivalent per-action confidence signal. Training proceeds over successive checkpoints (10k through 500k+ steps) so the evaluation stage can compare early, exploratory behaviour against late, tactically converged behaviour.

Actor-Critic Signal Logging Methodology (Feedback-Driven Extension) This stage implements the reviewer's Q-Value / Advantage-Aware Explanation RAG recommendation directly. Every time Module 2 selects an action, it doesn't just throw away the intermediate actor-critic computation the way a standard training loop normally would — instead it captures the full $\pi(a|s)$ vector, the scalar $V(s)$, and the shared penultimate-layer latent vector, and forwards this bundle to the telemetry logger in the same step. Where the original abstract's design logged only the sampled action and the resulting reward, the revised method logs the network's complete internal assessment of the state, including its assessment of actions it didn't take. This is what makes the later retrieval-and-explanation stage able to answer counterfactual questions instead of only descriptive ones.

Retrieval-Augmented Generation Methodology Telemetry records get embedded and stored in ChromaDB as they're produced. At query time, the user's natural-language question is embedded with the same embedding model and compared against the stored telemetry embeddings using cosine similarity, returning the top-k most relevant frames (usually the frame nearest the referenced timestamp plus a small surrounding window, so the LLM can see the state just before and after the decision in question). The retrieved numerical context, along with the relevant static game-rule entries, gets inserted into a structured prompt template that explicitly labels each number (e.g. " $V(s) = -0.34$ ", " $\pi(\text{Shoot}) = 0.08$ ", " $\pi(\text{Retreat}) = 0.71$ ") so the LLM isn't asked to guess meaning from unlabelled numbers. The LLM (a locally hosted Llama 3 / Mistral model via Ollama, with the OpenAI API available as a fallback) is instructed to answer using only the supplied figures and to explicitly compare the chosen action's probability and the state's value against the alternatives, which is the mechanism producing the counterfactual explanation requested in the feedback.

Counterfactual Explanation Methodology Where a purely descriptive RAG system would answer "why did the agent retreat?" by just restating the retrieved state ("the agent had low health and the opponent was nearby"), the counterfactual

method introduced in this revision goes one step further by explicitly contrasting the chosen action against its rejected alternatives, using the logged $\pi(a|s)$ and $V(s)$ figures. The prompt template used here is deliberately structured to force this comparison, asking the LLM to (i) state which action was chosen, (ii) state the probability the policy assigned to at least one plausible alternative action, (iii) state the value estimate for the current state, and (iv) connect these three figures into one coherent explanation. Since every figure quoted in step (iv) is copied verbatim from the retrieved log record rather than generated, the explanation stays fully traceable and avoids the black-box-explaining-a-black-box problem that would come up if we asked an ungrounded LLM to just guess at the agent's reasoning.

Evaluation Methodology We plan to evaluate the system on three fronts, once Modules 2 and 3 have actually been built and run:

- RL training performance tracked in TensorBoard — episodic reward, episode length, and policy/value loss curves across checkpoints — to confirm convergence from random to tactical behaviour.
- Vector database performance evaluated on retrieval accuracy (whether the top-k retrieved frames actually correspond to the timestamp/event a test question refers to) and retrieval latency (time from query submission to retrieved context being ready for the LLM prompt), so the sub-second explanation target from the project's key features can be verified quantitatively.
- Explanation quality checked for factual faithfulness by manually cross-checking a sample of generated explanations against the raw logged figures they cite, confirming that the quoted $V(s)$ and $\pi(a|s)$ values in the generated text match the underlying log exactly and that no unsupported claims sneak in.

None of these three evaluations can be run yet at the first-phase stage — they all depend on the agent actually being trained and the vector database actually being queried under load, which is exactly the "prior build" of Modules 2 and 3 described in Section IV. Figure 5 marks where the resulting TensorBoard curves and retrieval-latency table will go once that work has been done; we have deliberately left it as a placeholder here rather than showing invented numbers, since presenting simulated results as though they were measured would misrepresent where the project actually stands.

RAG-based explanation engine, and revised the telemetry design so it captures the actor-critic's internal reasoning — not just its final action — directly addressing the reviewer feedback from the abstract stage. Subsequent phases will report the trained agent's quantitative performance, the

VI. CONCLUSION

This concludes the First Phase submission. At this stage, the project's contribution is mainly architectural: we've laid out a three-module system that connects a trainable PPO agent to a

finalised RAG evaluation results referenced in Section V-B, and a live demonstration of the counterfactual explanation dashboard described in Sections III and IV-C.